Proxy

Season: 2 · Structural · Category: GoF · Level: Beginner · Build with Waqar

Today's Hook

- Today: Proxy as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Proxy structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Proxy
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/structural/proxy/LazyImageProxy.java
package com.buildwithwaqar.patterns.gof.structural.proxy;

interface Image {
    String display();
}

final class RealImage implements Image {
    private final String path;
    RealImage(String path) { this.path = path; }
    public String display() { return "showing:" + path; }
}

/** Virtual proxy — lazy-loads the real subject. */
public final class LazyImageProxy implements Image {
    private final String path;
    private RealImage real;

    public LazyImageProxy(String path) { this.path = path; }

    @Override
    public synchronized String display() {
        if (real == null) real = new RealImage(path);
        return real.display();
    }
```

Implementation (2)

```
    boolean loaded() { return real != null; }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.structural.proxy;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Proxy walkthrough ===");
    var proxy = new LazyImageProxy("/assets/thumb.png");
    System.out.println("loadedBefore=" + proxy.loaded());
    System.out.println(proxy.display());
    System.out.println("loadedAfter=" + proxy.loaded());
    System.out.println(proxy.display());
  }
}
```

Walkthrough Output

```
=== Proxy walkthrough ===
loadedBefore=false
showing:/assets/thumb.png
loadedAfter=true
showing:/assets/thumb.png
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Proxy
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Proxy → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.